

C++ MINI PROJECT PLAN

HOSTEL MANAGEMENT SYSTEM

PERSON1: FRONTEND

PLAN:

Frontend Development Plan

1. Technology Selection

1. HTML
2. CSS
3. JavaScript
4. Fetch API for communication with the backend
5. JSON data handling
6. Responsive and user-friendly web interface

2. Project Structure

1. Create separate frontend files for HTML, CSS, and JavaScript.
2. Organize student and administrator pages clearly.
3. Keep frontend files separate from the C++ backend.
4. Connect the frontend to the backend server through HTTP requests.

3. Login Page

1. Create a login interface for user ID and password.
2. Send login credentials to the backend.
3. Receive and process the login response.
4. Identify whether the logged-in user is a student or administrator.
5. Redirect the user to the appropriate dashboard.

4. Student Interface

1. Create the student dashboard.
2. Display student information.
3. Display room/allocation details.
4. Provide an interface for viewing hostel information.
5. Provide a complaint submission interface.

5. Administrator Interface

1. Create the administrator dashboard.
2. Provide interfaces for managing student records.
3. Provide interfaces for managing rooms.
4. Display room availability and vacancy information.
5. Provide room allocation management interface.
6. Provide complaint management interface.

6. Student Management UI

1. Create forms for adding student information.
2. Display student records in a readable format.
3. Provide edit and update controls.
4. Provide delete controls where required.
5. Connect all operations to the corresponding backend APIs.

7. Room Management UI

1. Create a room listing page.
2. Display room number, block, floor, capacity, and availability.
3. Show available rooms clearly.
4. Provide controls for updating room information through the backend.
5. Refresh room information after changes.

8. Room Allocation UI

1. Create an interface for assigning students to rooms.
2. Display current room allocation details.
3. Provide options to change or remove an allocation.
4. Update the displayed vacancy information after allocation changes.

9. Complaint Management UI

1. Create a complaint submission form for students.
2. Display submitted complaints and their status.
3. Provide administrator controls for viewing and updating complaint status.
4. Refresh complaint information after backend operations.

10. Backend API Integration

1. Use JavaScript fetch requests to communicate with the C++ backend.
2. Send and receive JSON data.
3. Connect login requests to the login API.
4. Connect student, room, allocation, and complaint pages to their respective APIs.
5. Handle successful and failed API responses on the frontend.

11. Frontend Testing

1. Test login with valid and invalid credentials.
2. Test student and administrator navigation.
3. Test forms and buttons for all major modules.
4. Test frontend-to-backend communication.
5. Test display of data returned by the backend.
6. Test error messages and failed requests.
7. Test the complete workflow from login to dashboard operations.

12. Frontend Styling and Usability

1. Apply consistent CSS styling across all pages.
2. Create clear navigation and readable layouts.
3. Make forms, tables, buttons, and dashboards easy to use.
4. Ensure the pages work properly on common screen sizes.
5. Keep the design simple and suitable for project evaluation.

13. Frontend Documentation

1. Document the frontend folder and file structure.
2. Document the purpose of each HTML, CSS, and JavaScript file.
3. Document the API calls used by the frontend.
4. Record frontend testing results.
5. Keep the code simple and understandable so that every module can be explained during the project evaluation.